1. 软件工程与软件过程

- **定义**：软件工程是从管理和技术两方面研究如何更好开发和维护软件的学科。
- **软件危机 6 问题**：无计划、需求不足、无规范、成本进度难控、质量无标准、维护困难。
- **软件过程 4 活动**：需求分析、设计和实现、测试、演进。
- **速记**：危机看问题，工程看解决；过程背“需设测演”。

2. 瀑布、螺旋、敏捷

- **瀑布模型**：线性顺序开发，阶段清楚，适合需求稳定项目。
- **瀑布缺点**：需求难一次说清、反馈晚、变更难、错误发现迟。
- **螺旋模型**：迭代开发 + 风险驱动，结合原型模型和瀑布模型特点。
- **敏捷开发**：快速交付、客户紧密参与、适合需求变化快的中小项目。
- **速记**：瀑布重顺序，螺旋重风险，敏捷重反馈。

3. 结构化设计核心

- **模块独立性**：模块只完成单一子功能，接口简单。
- **设计目标**：低耦合、高内聚。
- **耦合排序**：非直接 < 数据 < 标记 < 控制 < 外部 < 公共 < 内容。
- **内聚排序**：巧合 < 逻辑 < 时间 < 过程 < 通信 < 顺序 < 信息 < 功能。
- **速记**：耦合越低越好，内聚越高越好。

4. 需求分析

- **任务**：弄清用户对软件系统的确切要求，并写成需求规格说明书。
- **原则**：准确、一致、清晰、无二义性。
- **步骤**：调查现实环境是怎么运行 -> 把不重要的细节去掉，抓住系统最核心的逻辑 -> 观察现在的系统与目标系统的差别，将目标系统描绘清楚-> 把目标系统该有的功能和要求补完整，写成需求说明书 -> 检查和评审，直到这份需求说明符合用户需求。
- **速记**：先调研，再抽象，再建模，再完善，再评审。

5. 需求规格说明书与工具

- **SRS 内容**：引言、数据描述、功能描述、性能描述、质量保证。
- **DFD**：表达目标系统逻辑模型。
- **数据字典**：统一定义数据，保证一致性。
- **判定表/判定树**：表达复杂判断逻辑。
- **速记**：需求阶段看 DFD，数据描述靠数据字典。

6. 软件设计与详细设计工具

- **软件设计**：把需求转化为软件表示。
- **概要设计**：定体系结构和数据结构。
- **详细设计**：定详细数据结构和算法。
- **常用表达工具**：程序流程图、N-S 图、PAD、伪代码。
- **易混点**：`HIPO图` 常用于层次与输入输出描述，不是常考的详细设计主工具。

7. 程序流程图与 N-S 图

- **流程图**：强调流程走向，符号常考。
- **N-S 图**：不用流程线，用矩形体现结构化程序。
- **三种基本结构**：顺序、选择、循环。
- **速记**：流程图看“流向”，N-S 图看“结构”。

8. 面向对象基础

- **对象**：属性 + 操作的统一体。
- **类**：同类对象的抽象，对象是类的实例。
- **封装**：隐藏内部实现细节。
- **继承**：子类自动拥有父类属性和服务。
- **多态**：同一消息，多种实现；C++ 中靠虚函数实现。
- **速记**：封装藏细节，继承接家产，多态多实现。

9. UML 图速记

- **用例图**：看系统功能。
- **类图**：看静态结构。
- **时序图**：看对象交互顺序。
- **状态图**：看状态变化。
- **部署图**：看物理布局。
- **组件图**：看软件组件关系。
- **速记**：功能用例，结构类图，交互时序，变化状态，落地部署，构件组件。

10. 类图关系

- **关联**：最基本关系。
- **泛化**：一般与特殊，父子类关系。
- **聚合**：整体与部分，部分可独立存在。
- **组合**：整体与部分，部分不能独立存在。
- **依赖**：弱相关的使用关系。
- **易混点**：聚合“可分开”，组合“共存亡”。

11. 软件测试总框架

- **测试目的**：发现错误，不是证明程序无错。
- **测试对象**：需求文档、设计文档、源程序。
- **四步骤**：单元测试、组装测试、确认测试、系统测试。
- **速记**：先单元，再组装，再确认，后系统。

12. 黑盒与白盒

- **黑盒测试**：不看内部代码，按功能和输入输出设计用例。
- **黑盒方法**：等价类划分、边界值分析。
- **白盒测试**：看内部逻辑结构，检查语句、条件、判定、路径。
- **白盒覆盖**：语句覆盖、判定覆盖、条件覆盖、判定-条件覆盖、条件组合覆盖、路径覆盖。
- **速记**：黑盒看功能，白盒看路径。

13. 静态测试、动态测试与测试辅助模块

- **静态测试**：不运行程序，靠检查、评审、走查发现问题。
- **动态测试**：运行程序，通过输入输出发现问题。
- **驱动模块**：代替被测模块的调用模块。
- **桩模块**：代替被测模块调用的下层模块。
- **速记**：上面缺调用写驱动，下面缺被调写桩。

14. 组装测试与确认测试

- **组装测试**：把单元测试通过的模块组合起来测试。
- **方式**：一次性组装、增殖式组装。
- **增殖式**：自顶向下、自底向上、混合组装。
- **确认测试**：验证软件是否满足用户需求。
- **系统测试**：软件和硬件、外设、数据、人员一起在真实环境中综合测试。

15. 软件维护与可维护性

- **软件维护定义**：软件交付后，为改错或满足新需求而修改软件的过程。
- **维护类型**：改正性、适应性、完善性、预防性。
- **引起维护原因**：发现错误、环境变化、用户新需求。
- **可维护性指标**：可理解性、可靠性、可测试性、可修改性、可移植性、效率、可使用性。
- **速记**：改错是改正，换环境是适应，加功能是完善，防未来是预防。

16. 三层架构与 MVC

- **三层架构**：界面层、业务逻辑层、数据访问层。
- **MVC**：Model、View、Controller。
- **易混点**：三层讲职责分层，MVC 讲交互组织；两者可同时存在。

17. 移动应用开发

- **五阶段**：需求分析、设计、开发、稳定、部署。
- **移动应用特点**：重用户体验、环境复杂、技术难度高。
- **设计阶段**：体系结构、导航、界面、内容设计。
- **速记**：需设开发稳部。

18. 移动应用测试

- **常见测试**：用户体验、兼容性、性能、连接性、安全性、自然环境、认证测试。
- **用户体验重点**：手势、虚拟键盘、语音输入识别、警报与异常条件。
- **原则**：性能、GUI、兼容性测试尽量用真实设备。

19. 设计文档与结构图

- **概要设计说明书**：总体结构、模块功能接口、数据结构设计。
- **详细设计说明书**：模块逻辑流程、输入输出、接口说明。
- **结构图 SC**：描述软件模块结构和调用关系。
- **典型结构**：变换型结构、事务型结构。

20. 最后背诵抓手

- **模型题**：重点背瀑布、螺旋、敏捷的区别。
- **设计题**：重点背需求分析步骤、低耦合高内聚、概要/详细设计区别。
- **测试题**：重点背黑盒/白盒、四种测试步骤、驱动/桩模块。
- **面向对象题**：重点背封装、继承、多态、UML 图作用。
- **维护题**：重点背四类维护和可维护性指标。